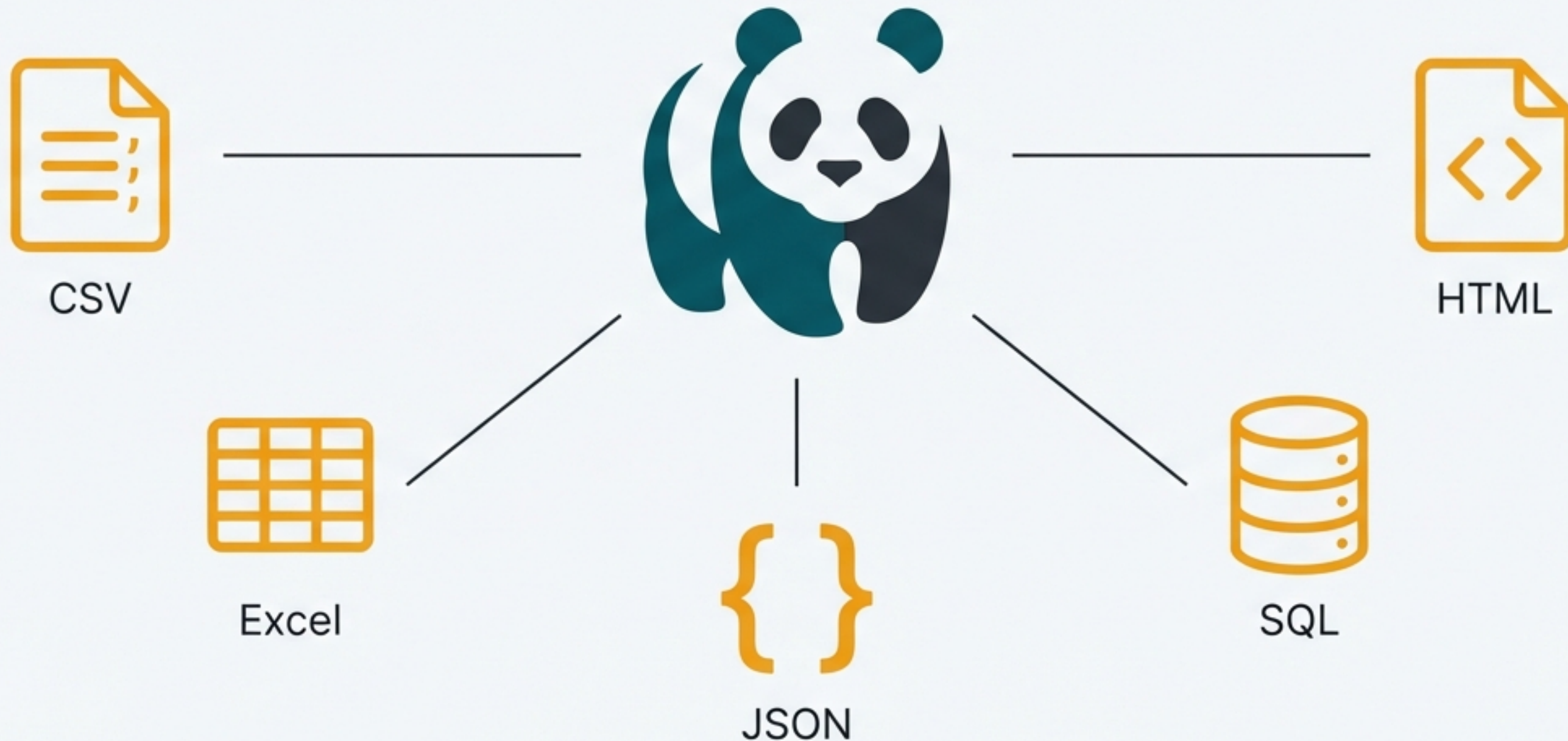


Domina la Entrada y Salida de Datos con Pandas

Una guía práctica para importar, limpiar y exportar datos como un experto.



El primer desafío: El mundo real de los datos.

Antes de cualquier análisis, modelo o visualización, nos enfrentamos a una realidad: los datos están **dispersos** en múltiples formatos. Gran parte de nuestro tiempo se dedica a la **limpieza**, y el **primer** paso es **siempre** la **importación**.

Conocer nuestros datos es fundamental.

```
Name;Age;City
Alice;28;New York
Bob;thirty;San Francisco
Charlie;22;London
David;;Berlin
```

```
Date: Mar. 15, 2023 | Value: '4500'
```

```
{
  "user": {
    "id": 123,
    "profile": {
      "name": "Eve",
      "preferences": {
        "theme": "dark",
        "notifications":
      }
    }
  }
}
```

```
<table>
  <tr>
    <td>ID</td>
    <td>Product</td>
  </tr>
  <tr>
    <td>001</td>
    <td>Widget A</td>
  </tr>
</table>
```


La solución de Pandas: Un comando para cada formato.

Pandas ofrece un conjunto de funciones intuitivas y potentes, ``read_*``, para importar datos desde prácticamente cualquier fuente a un DataFrame limpio y estructurado.

Función	Descripción
<code>read_csv</code>	Crea dataframe con los datos que lee de un archivo tipo CSV. El delimitador por defecto es la coma.
<code>read_excel</code>	Lee datos tabulares de un archivo XLS o XLSX de Excel.
<code>read_sql</code>	Lee los resultados de una consulta SQL (utilizando la biblioteca SQLAlchemy).
<code>read_json</code>	Lee datos desde un archivo, URL o cadena de texto JSON.
<code>read_html</code>	Lee todas las tablas que encuentra en el documento HTML dado.
<code>read_clipboard</code>	Lee datos del portapapeles.
<code>read_fwf</code>	Lee archivos de texto con columnas de ancho fijo, sin delimitadores.
<code>read_xml</code>	Lee una tabla o datos de un archivo SQL.
<code>read_stata</code>	Lee conjunto de datos de un formato de archivo Stata (.dta).

Empecemos por lo básico: Importar un archivo Excel.

Challenge

Tienes una lista de clientes en un archivo ``Lista-de-clientes.xlsx``. Necesitas cargarla en Python para analizarla.

Solution

```
import pandas as pd

empleados = pd.read_excel("Lista-de-clientes.xlsx")
print(empleados)
```

Result

	ID	Grupo de clientes
0	C0001	...
1	C0002	...
2	C0003	...
3	C0004	...
4	C0005	...
..	...	...
994	C0995	...
995	C0996	...
996	C0997	...
997	C0998	...
998	C0999	...

[999 rows x 9 columns]

Un universo de formatos al alcance de tu mano.

Desde el Portapapeles

```
portapapeles = pd.read_clipboard()
```

	ID	Nombre completo	Fecha de nacimiento
0	C0001	Leandra Anna Malo Alba	08/12/1984
1	C0002	Severo Granados Iglesia	12/08/1986
2	C0003	Lucho Andreu Amat	16/04/1990
3	C0004	Matías Mauricio Castillo Barrera	02/12/1996
4	C0005	Mauricio Guijarro Castelló	14/05/1984
5	C0006	Isaura Leyre Avilés Pelayo	18/04/1987
6	C0007	Soraya Morera-Lago	27/11/1990
7	C0008	Victoriano Tapia-Cabanillas	08/03/1981
8	C0009	Nidia Saez Campoy	04/04/1980
9	C0010	Teófila Villanueva Molina	04/01/1987
10	C0011	Trini de Alberdi	04/12/1990
11	C0012	Dani Baena	20/10/1977
12	C0013	Angelina de Arregui	21/02/2000
13	C0014	Samuel de Carranza	16/05/1999
14	C0015	Jacinta Montenegro Garcés	13/03/1994
15	C0016	Lisandro Delgado Nadal	18/04/1980

Desde JSON

```
json = pd.read_json("datos.json")
```

	ID	Name
0	101	Alice
1	102	Bob
2	103	Charlie
3	104	David
4	105	Eva

Desde HTML

```
html = pd.read_html("simple-table.html")
```

	0	...	3
0	Hi, I'm your first cell.	...	I'm your fourth cell.
1	Second row, first cell.	...	Cell 4.

Conexión directa: De la base de datos a tu DataFrame.

Pandas puede ejecutar consultas SQL y cargar los resultados directamente en un DataFrame, utilizando librerías como SQLAlchemy para la conexión.

```
import pandas as pd
import sqlite3

# Nos conectamos a la base de datos
con = sqlite3.connect('Our DataBase')

# Leemos la tabla directamente con Pandas
sql_query = pd.read_sql('SELECT * FROM COURSES', con)
print(sql_query)
```

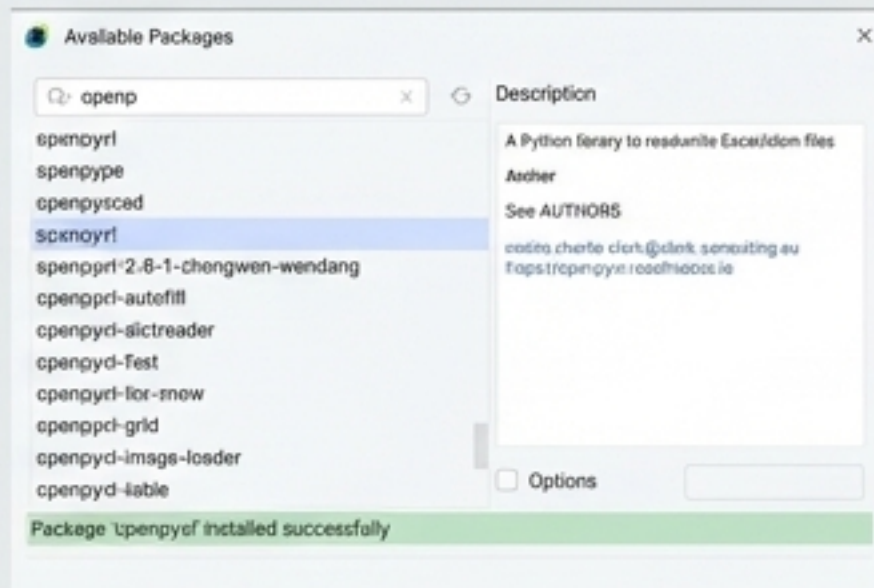
	course_id	course_name	fee	duration	discount
0	1	Spark	25000	50 Days	2000
1	2	Pandas	20000	35 Days	1000
2	3	Java	15000	35 Days	800
3	4	Python	15000	30 Days	500
4	5	PHP	28000	30 Days	800

Preparando el terreno: Las librerías que potencian a Pandas.

Para algunos formatos, Pandas se apoya en librerías especializadas. Asegúrate de tenerlas instaladas en tu entorno.

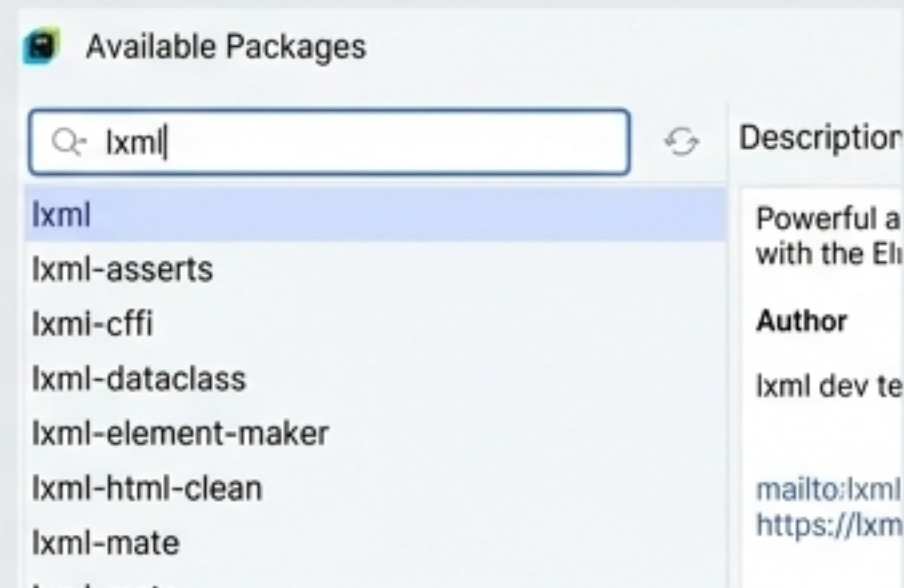
Para Excel (XLSX)

openpyxl



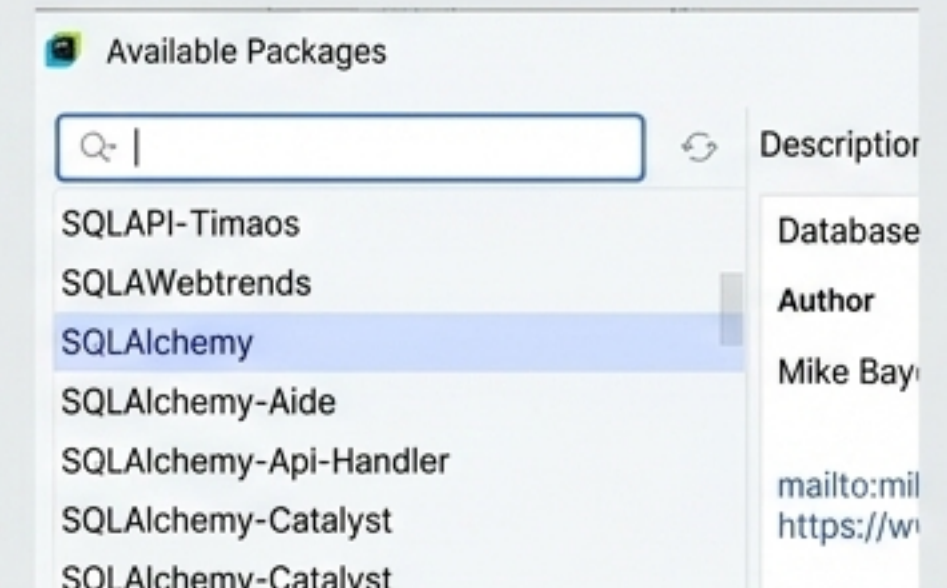
Para HTML/XML

lxml



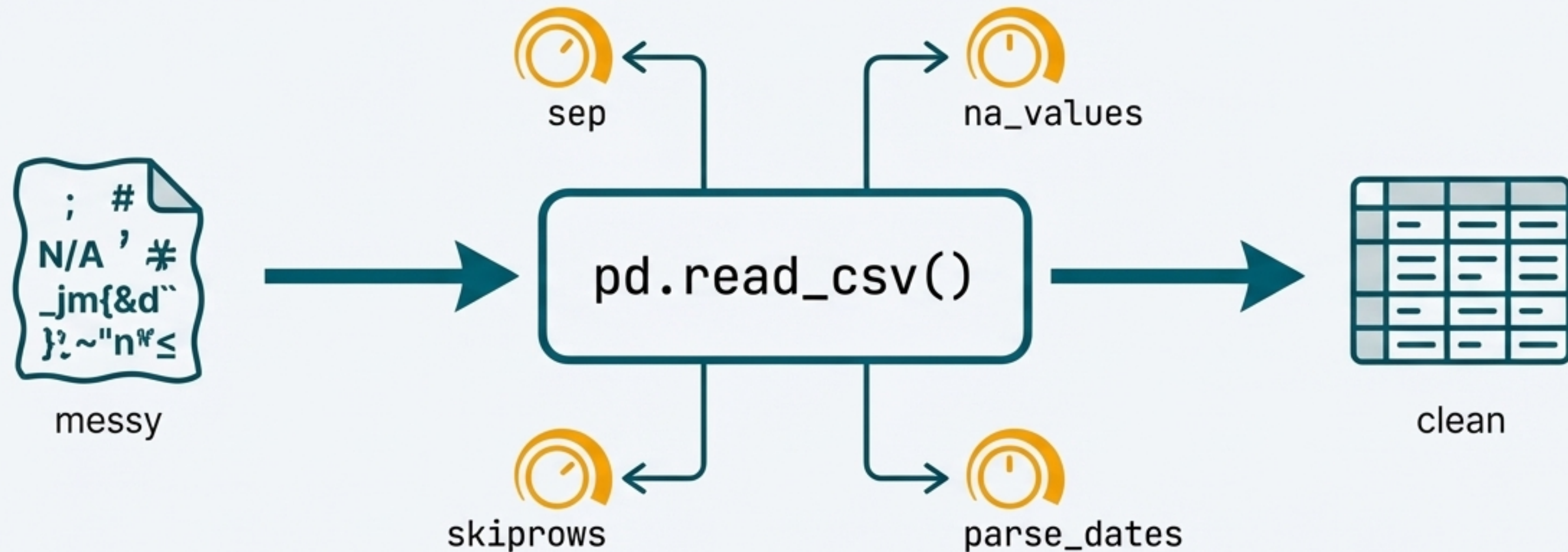
Para SQL

SQLAlchemy



El mundo real no es perfecto. Tus herramientas de importación sí.

¿Qué pasa si tu CSV no usa comas como separador? ¿Si tienes filas que ignorar al principio? ¿O si las fechas son solo texto? Para estos casos, Pandas ofrece un completo panel de control a través de sus argumentos opcionales.



Tu panel de control para la importación de datos.

Argumento	Descripción
<code>sep</code> o <code>delimiter</code>	Caracteres o expresión regular que especifican el separador de campos.
<code>header</code>	Especifica el número de fila que contiene los nombres de las columnas.
<code>index_col</code>	Especifica el número o nombres de columna que se debe usar como índice de fila.
<code>skiprows</code>	Lista de número de líneas que se deben ignorar al leer el archivo.
<code>usecols</code>	Especifica una lista de nombres de columna que se deben leer del archivo.
<code>na_values</code>	Secuencia de valores que se deben identificar como valores NA.
<code>parse_dates</code>	Especifica una lista de nombres de columnas que se deben interpretar como fechas.
<code>dtype</code>	Especifica el tipo de datos de cada columna.
<code>nrows</code>	Número de filas que se leen desde el principio del archivo.
<code>decimal</code>	Especifica el separador de los decimales en números. Por defecto es '.'.
<code>thousands</code>	Especifica el separador de los miles para números.

Personalización en acción: Enfocándonos en lo que importa.

Trabajando con el archivo `InventoryLot\_v4.csv`.

🔪 Caso Práctico 1: Seleccionar Columnas Específicas

El archivo tiene muchas columnas, pero solo necesitas la cantidad y el valor, usando el código de lote como índice.

```
lectura2 = pd.read_csv("InventoryLot_v4.csv",
                        index_col="lotCode",
                        usecols=["lotCode", "quantity", "value"])

print(lectura2)
```

	quantity	value
lotCode		
521033	25	31250.0
3282509	20	25000.0
862421	3	3750.0
9282139	35	43750.0
240235	43	53750.0
...	...	...
8774537	118	151866.0
9695355	323	470288.0
8012120	273	227409.0
3516489	259	267288.0
4216598	27	26352.0

[154 rows x 2 columns]

🔪 Caso Práctico 2: Omitir Filas y Limitar la Lectura

El archivo es muy grande. Quieres saltarte las primeras 100 filas de datos y leer solo las 10 siguientes para una inspección rápida.

```
lectura3 = pd.read_csv("InventoryLot_v4.csv",
                        skiprows=100,
                        nrows=10)

print(lectura3)
```

101	7900014	LT-1	ITEM	387	...	7353	USD	https://foo.com	2022-01-01T00:00:00
102	7900015	LT-1	ITEM	155	...	465	USD	https://foo.com	2022-01-01T00:00:00
103	9664144	LT-1	ITEM	37	...	888	USD	https://foo.com	2022-01-01T00:00:00
104	9665306	LT-1	ITEM	233	...	699	USD	https://foo.com	2022-01-01T00:00:00
105	2530003	LT-1	ITEM	214	...	276660	USD	https://foo.com	2022-01-01T00:00:00
106	2530020	LT-1	ITEM	455	...	536900	USD	https://foo.com	2022-01-01T00:00:00
107	7903986	LT-1	ITEM	228	...	225720	USD	https://foo.com	2022-01-01T00:00:00
108	8222072	LT-1	ITEM	44	...	63888	USD	https://foo.com	2022-01-01T00:00:00
109	83008200	LT-1	ITEM	116	...	278168	USD	https://foo.com	2022-01-01T00:00:00
110	83008300	LT-1	ITEM	109	...	500528	USD	https://foo.com	2022-01-01T00:00:00
110	83008400	LT-1	ITEM	177	...	978810	USD	https://foo.com	2022-01-01T00:00:00

[10 rows x 15 columns]

Personalización en acción: Limpiando los datos desde el origen.

Trabajando con el archivo `InventoryLot\_v4.csv`.

Caso Práctico 1: Manejar Valores Nulos No Estándar

El archivo usa 'N/A' y cadenas vacías para representar datos faltantes, y quieres que Pandas los reconozca.

```
lectura4 = pd.read_csv("InventoryLot_v4.csv",
                      na_values=["N/A", "Desconocido", ""],
                      keep_default_na=False)
print(lectura4.info())
```

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 154 entries, 0 to 153
Data columns (total 15 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   product.partNumber                   154 non-null    object
 1   location.locationIdentifier          154 non-null    object
 2   inventoryType                       154 non-null    object
 3   quantity                           154 non-null    int64
 4   quantityUnits                      154 non-null    object
 5   expirationDate                     154 non-null    object
 6   inventoryParentType                152 non-null    object
 7   class                              153 non-null    object
 8   segment                            151 non-null    object
 9   lotCode                            154 non-null    int64
10   status                             154 non-null    object
11   value                              152 non-null    float64
12   valueCurrency                      154 non-null    object
13   sourceLink                         154 non-null    object
14   storageDate                       154 non-null    object
dtypes: float64(1), int64(2), object(12)
memory usage: 18.2+ KB
None
```

Caso Práctico 2: Convertir Columnas a Fechas

La columna `expirationDate` está almacenada como texto, pero necesitas que sea un objeto de fecha para poder hacer cálculos con ella.

```
lectura5 = pd.read_csv("InventoryLot_v4.csv",
                      parse_dates=["expirationDate"])
print(lectura5["expirationDate"])
```

```
0    2022-12-31 00:00:00
1    2022-12-31 09:00:00
2    2022-12-31 10:00:00
3    2022-12-31 00:00:00
4    2022-12-31 05:00:00
...
149  2022-12-31 00:00:00
150  2022-12-31 00:00:00
151  2022-12-31 00:00:00
152  2022-12-31 00:00:00
153  2022-12-31 00:00:00
Name: expirationDate, Length: 154, dtype: datetime64[ns]
```


La última milla: Exportando tus resultados al mundo.

El trabajo no termina hasta que compartes tus resultados. Con los métodos ``to_*``, puedes guardar tus DataFrames limpios y procesados en el formato que necesites, con la misma facilidad con la que los importaste.

Método	Descripción
<code>to_csv</code>	Escribe el objeto DataFrame en un archivo separado por comas (csv).
<code>to_excel</code>	Copia el objeto en un archivo de Excel.
<code>to_sql</code>	Escribe los datos del DataFrame en una base de datos SQL.
<code>to_json</code>	Convierte el objeto en un string JSON.
<code>to_html</code>	Representa el DataFrame como una tabla HTML.
<code>to_clipboard</code>	Copia el objetos en el portapapeles del sistema.
<code>to_parquet</code>	Escribe el DataFrame en el formato Parquet.
<code>to_xml</code>	Representa el DataFrame en un documento XML.

Exportando con precisión y control.

CSV Parcial sin Índice

Exportar solo 3 columnas a un nuevo CSV, sin la columna de índice automática de Pandas.

```
lectura1[["product.partNumber", "quantity", "expirationDate"]].to_csv("exportado_parcial.csv", index=False)
```

Excel con Columnas Seleccionadas

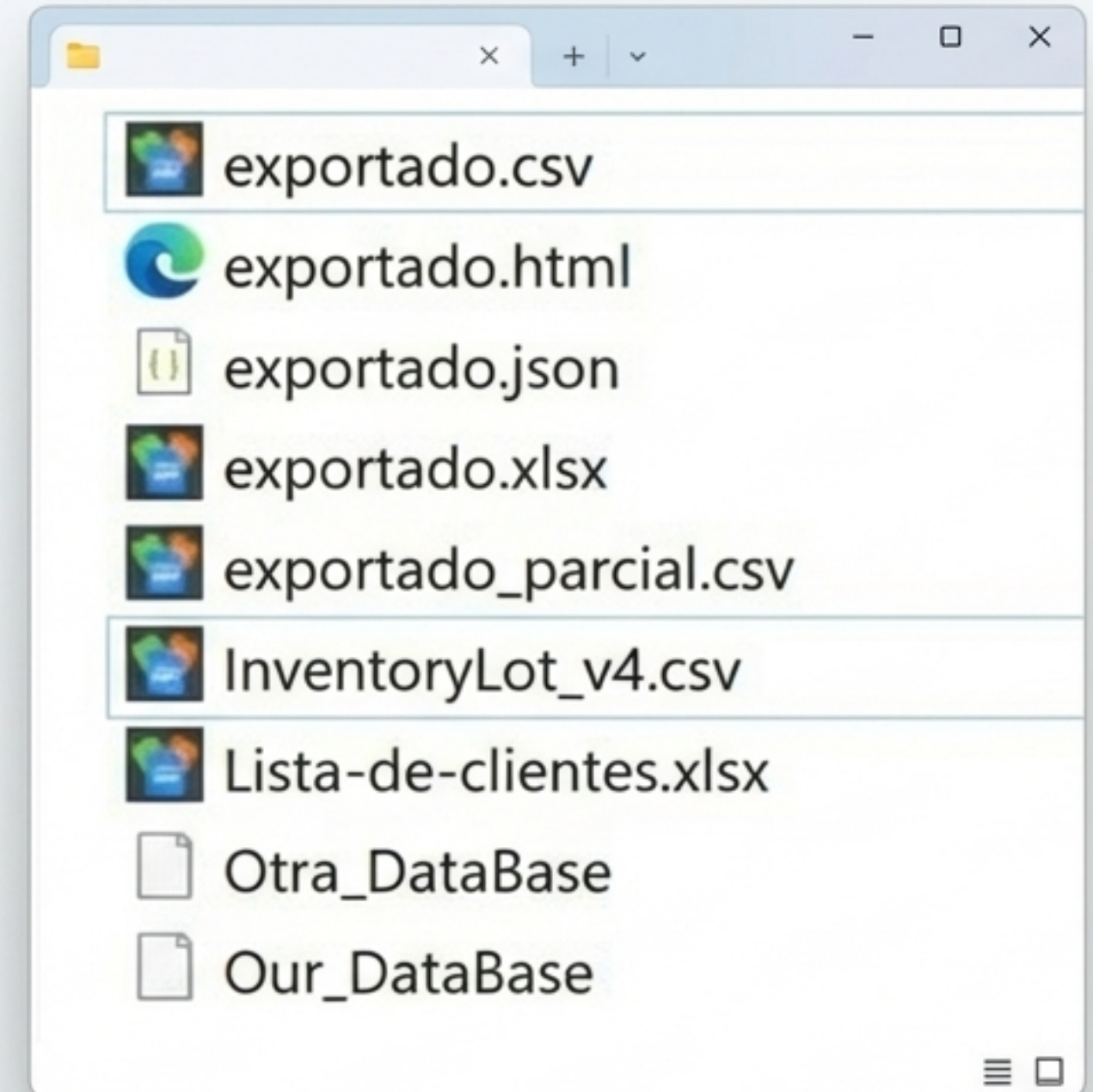
Guardar un subconjunto de datos de clientes en un nuevo archivo de Excel.

```
empleados[["ID", "Nombre completo", "Grupo de clientes"]].to_excel("exportado.xlsx", index=False)
```

JSON Formateado

Exportar las primeras 5 filas a un archivo JSON legible para humanos.

```
json.head(5).to_json("exportado.json",  
                    orient="records", indent=4)
```



Tu Chuleta Definitiva de I/O con Pandas.

LEER DATOS (Importar)

`pd.read_*("fuente", **kwargs)`

-  **read_csv()**: Leer un archivo separado por comas.
-  **read_excel()**: Leer un archivo de Excel.
-  **read_sql()**: Leer datos de una base de datos SQL.
-  **read_json()**: Leer un archivo o string JSON.
-  **read_html()**: Leer tablas de una página HTML.
-  **read_clipboard()**: Leer datos del portapapeles.

Argumentos Clave

`sep, header, usecols, parse_dates, na_values`

ESCRIBIR DATOS (Exportar)

`df.to_*("destino", **kwargs)`

-  **to_csv()**: Escribir el DataFrame en un archivo CSV.
-  **to_excel()**: Escribir el DataFrame en un archivo de Excel.
-  **to_sql()**: Escribir el DataFrame en una base de datos SQL.
-  **to_json()**: Convertir el DataFrame en un string o archivo JSON.
-  **to_html()**: Representar el DataFrame como una tabla HTML.
-  **to_clipboard()**: Copiar el DataFrame al portapapeles.

Argumentos Clave

`index=False, columns, orient, if_exists`